

第 5 章

语句

导读

本章介绍了几种典型的 C++ 语句，它们分别是：

- 空语句和语句块
- 条件语句 (if、switch)
- 迭代语句 (while、do-while、for、范围 for)
- 跳转语句 (break、continue、goto)

此外还介绍了与语句密切相关的知识：语句的作用域和异常处理机制。本章的练习包含两种题型：一种是考查读者对本章介绍知识的掌握程度，尤其是对核心概念的理解是否准确；另一种是基于所学的语句语法规则，编写解决实际问题的程序。

对于同一类语句来说，不同的语法形式常常可以互相转换。读者应该理解并练习 if 语句和 switch 语句如何互相转换；while 语句、do-while 语句和 for 语句如何互相转换。

练习 5.1：什么是空语句？什么时候会用到空语句？

【出题思路】

理解空语句的形式和用法。

【解答】

空语句是最简单的语句，空语句由一个单独的分号构成。如果在程序的某个地方，语法上需要一条语句但是逻辑上不需要，此时应该使用空语句，空语句什么也不做。

一种常见的情况是，当循环的全部工作在条件部分就可以完成时，我们通常会用到空语句。使用空语句时最好加上注释，从而令代码的读者知道这条语句是有意省略内容的。

练习 5.2: 什么是块? 什么时候会用到块?**【出题思路】**

理解块的形式和用法。

【解答】

块是指用花括号括起来的语句和声明的序列, 也称为复合语句。一个块就是一个作用域, 在块中引入的名字只能在块内部以及嵌套在块中的子块里访问。如果在程序的某个地方, 语法上需要一条语句, 但是逻辑上需要多条语句, 此时应该使用块。块不需要以分号结束。

例如, 循环体必须是一条语句, 但是我们通常需要在循环体内做很多事情, 此时就应该把多条语句用花括号括起来, 从而把语句序列转变成块。

练习 5.3: 使用逗号运算符 (参见 4.10 节, 第 140 页) 重写 1.4.1 节 (第 10 页) 的 while 循环, 使它不再需要块, 观察改写之后的代码的可读性提高了还是降低了。

【出题思路】

使用连续的逗号运算符可以把多条语句合并为一条, 这一点与块的作用类似。但是一般来说, 直接使用块在程序的可读性上更有优势。

【解答】

原文的 while 循环使用了块, 其形式是:

```
while ( val <= 10 )
{
    sum += val;
    ++val;
}
```

利用逗号运算符改写之后的形式如下所示:

```
while ( val <= 10 )
    sum += val, ++val;
```

很明显, 改写之后的代码不够清晰, 可读性降低了。

练习 5.4: 说明下列例子的含义, 如果存在问题, 试着修改它。

- (a) while (string::iterator iter != s.end()) { /* ... */ }
- (b) while (bool status = find(word)) { /* ... */ }
if (!status) { /* ... */ }

【出题思路】

我们可以在 if、switch、while 和 for 语句的控制结构内定义变量。定义在控制结构当中的变量只在相应语句的内部可见, 一旦语句结束, 变量也就超出其作用范围了。如果其他代码也需要访问控制变量, 则变量必须定义在语句的外部。

【解答】

(a) 是非法的, 它的原意是希望在 while 语句的控制结构当中定义一个

string::iterator 类型的变量 iter，然后判断 iter 是否到达了 s 的末尾，只要还没有到达末尾就执行循环体的内容。但是该式把变量的定义和关系判断混合在了一起，如果要使用 iter 与其他值比较，必须首先为 iter 赋初值。修改后的程序应该是：

```
string::iterator iter=s.begin();
while (iter != s.end())
{
    ++iter;
    /*...*/
}
```

(b)是非法的，变量 status 定义在 while 循环控制结构的内部，其作用域仅限于 while 循环。if 语句已经位于 while 循环的作用域之外，status 在 if 语句内是一个未命名的无效变量。要想在 if 语句中继续使用 status，需要把它定义在 while 循环之前。修改后的程序应该是：

```
bool status;
while (status = find(word)) { /* ... */ }
if (!status) { /* ... */ }
```

练习 5.5：写一段自己的程序，使用 if-else 语句实现把数字成绩转换成字母成绩的要求。

【出题思路】

练习 if-else 语句的基本语法结构。

【解答】

满足题意的程序如下所示：

```
#include <iostream>

using namespace std;

int main()
{
    int grade;
    cout << "请输入您的成绩：";
    cin >> grade;
    if(grade < 0 || grade > 100)
    {
        cout << "该成绩不合法" << endl;
        return -1;
    }
    if(grade == 100)    // 处理满分的情况
    {
        cout << "等级成绩是：" << "A++" << endl;
        return -1;
    }
    if(grade < 60)      // 处理不及格的情况
    {
        cout << "等级成绩是：" << "F" << endl;
```

```

        return -1;
    }
    int iU = grade / 10;    // 成绩的十位数
    int iT = grade % 10;    // 成绩的个位数
    string score, level, lettergrade;
    // 根据成绩的十位数字确定 score
    if(iU == 9)
        score = "A";
    else if(iU == 8)
        score = "B";
    else if(iU == 7)
        score = "C";
    else
        score = "D";
    // 根据成绩的个位数字确定 level
    if(iT < 3)
        level = "-";
    else if(iT > 7)
        level = "+";
    else
        level = "";
    // 累加求得等级成绩
    lettergrade = score + level;
    cout << "等级成绩是: " << lettergrade << endl;

    return 0;
}

```

练习 5.6: 改写上一题的程序, 使用条件运算符 (参见 4.7 节, 第 134 页) 代替 if-else 语句。

【出题思路】

条件运算符可以实现与 if-else 语句类似的功能, 当有嵌套的条件表达式时, 注意其满足右结合律。

【解答】

改写了 score 和 level 的求值形式, 得到满足题意的程序如下所示:

```

#include <iostream>

using namespace std;

int main()
{
    int grade;
    cout << "请输入您的成绩: ";
    cin >> grade;
    if(grade < 0 || grade > 100)
    {
        cout << "该成绩不合法" << endl;
        return -1;
    }
}

```

```

if(grade == 100)    // 处理满分的情况
{
    cout << "等级成绩是: " << "A++" << endl;
    return -1;
}
if(grade < 60)      // 处理不及格的情况
{
    cout << "等级成绩是: " << "F" << endl;
    return -1;
}
int iU = grade / 10;    // 成绩的十位数
int iT = grade % 10;    // 成绩的个位数
string score, level, lettergrade;
// 根据成绩的十位数字确定 score
score = (iU == 9) ? "A"
        : (iU == 8) ? "B"
        : (iU == 7) ? "C" : "D";
// 根据成绩的个位数字确定 level
level = (iT < 3) ? "-"
        : (iT > 7) ? "+" : "";
// 累加求得等级成绩
lettergrade = score + level;
cout << "等级成绩是: " << lettergrade << endl;

return 0;
}

```

练习 5.7: 改正下列代码段中的错误。

- (a) if (ival1 != ival2)
 ival1 = ival2
 else ival1 = ival2 = 0;
- (b) if (ival < minval)
 minval = ival;
 occurs = 1;
- (c) if (int ival = get_value())
 cout << "ival = " << ival << endl;
 if (!ival)
 cout << "ival = 0\n";
- (d) if (ival = 0)
 ival = get_value();

【出题思路】

理解 if 语句的语法规则。

【解答】

(a) if 语句的循环体应该是一条语句，需要以分号结束，程序修改为：

```

if (ival1 != ival2)
    ival1 = ival2;

```

```
else ival1 = ival2 = 0;
```

(b) if 语句的循环体只能是一条语句, 本题从逻辑上来说需要做两件事, 一是修改 minval 的值, 二是重置 occurs 的值, 所以必须把这两条语句放在一个块里。程序修改为:

```
if (ival < minval)
{
    minval = ival;
    occurs = 1;
}
```

(c) ival 是定义在 if 语句中的变量, 其作用域仅限于第一个 if 语句, 要想在第二个 if 语句中也使用它, 就必须把它定义在两个 if 语句的外部。程序修改为:

```
int ival;
if (ival = get_value())
    cout << "ival = " << ival << endl;
if (!ival)
    cout << "ival = 0\n";
```

(d) 程序的原意是判断 ival 的值是否是 0, 原题使用赋值运算符的结果是把 0 赋给了 ival, 然后检验 ival 的值, 这样使得条件永远不会满足。程序修改为:

```
if (ival == 0)
    ival = get_value();
```

练习 5.8: 什么是“悬垂 else”? C++语言是如何处理 else 子句的?

【出题思路】

理解悬垂 else 的定义, 以及 C++ 是如何处理悬垂 else 的。

【解答】

悬垂 else 是指当程序中的 if 分支多于 else 分支时, 如何为 else 寻找与之匹配的 if 分支的问题。

C++ 规定, else 与离它最近的尚未匹配的 if 匹配, 从而消除了二义性。

练习 5.9: 编写一段程序, 使用一系列 if 语句统计从 cin 读入的文本中有多少元音字母。

【出题思路】

if 语句和 switch 语句是分支语句的两种形式, 一般来说可以互相转化。

【解答】

满足题意的程序如下所示:

```
#include <iostream>
using namespace std;

int main()
{
    unsigned int vowelCnt = 0;
    char ch;
    cout << "请输入一段文本: " << endl;
```

```

while(cin >> ch)
{
    if(ch == 'a')
        ++vowelCnt;
    if(ch == 'e')
        ++vowelCnt;
    if(ch == 'i')
        ++vowelCnt;
    if(ch == 'o')
        ++vowelCnt;
    if(ch == 'u')
        ++vowelCnt;
}
cout << "您输入的文本中共有 " << vowelCnt << "个元音字母" << endl;
return 0;
}

```

练习 5.10: 我们之前实现的统计元音字母的程序存在一个问题：如果元音字母以大写形式出现，不会被统计在内。编写一段程序，既统计元音字母的小写形式，也统计大写形式，也就是说，新程序遇到'a'和'A'都应该递增 aCnt 的值，以此类推。

【出题思路】

要实现本题的要求，只需更新 switch 语句的 case 分支即可。

【解答】

本题的关键是在适当位置添加 break; 语句。其中，表示同一个字母的大小写的 case 标签之间不写 break;，因为它们要做的操作是一样的；而不同字母的 case 分支最后要写上 break;。满足题意的程序如下所示：

```

#include <iostream>
using namespace std;

int main()
{
    unsigned int aCnt = 0, eCnt = 0, iCnt = 0, oCnt = 0, uCnt = 0;
    char ch;
    cout << "请输入一段文本: " << endl;
    while(cin >> ch)
    {
        switch(ch)
        {
            case 'a':
            case 'A':
                ++aCnt;
                break;
            case 'e':
            case 'E':
                ++eCnt;
                break;
            case 'i':
            case 'I':
                ++iCnt;

```

```

        break;
    case 'o':
    case 'O':
        ++oCnt;
        break;
    case 'u':
    case 'U':
        ++uCnt;
        break;
    }
}

cout << "元音字母 a 的数量是: " << aCnt << endl;
cout << "元音字母 e 的数量是: " << eCnt << endl;
cout << "元音字母 i 的数量是: " << iCnt << endl;
cout << "元音字母 o 的数量是: " << oCnt << endl;
cout << "元音字母 u 的数量是: " << uCnt << endl;
return 0;
}

```

练习 5.11: 修改统计元音字母的程序, 使其也能统计空格、制表符和换行符的数量。

【出题思路】

继续扩充 case 标签即可。其中, 读入字符的语句应该使用 `cin.get(ch)`, 而不能使用 `>>`, 因为后者会忽略本题所要统计的特殊符号。

【解答】

满足题意的程序如下所示:

```

#include <iostream>
using namespace std;

int main()
{
    unsigned int aCnt = 0, eCnt = 0, iCnt = 0, oCnt = 0, uCnt = 0;
    unsigned int spaceCnt = 0, tabCnt = 0, newlineCnt = 0;
    char ch;
    cout << "请输入一段文本: " << endl;
    while(cin.get(ch))
    {
        switch(ch)
        {
            case 'a':
            case 'A':
                ++aCnt;
                break;
            case 'e':
            case 'E':
                ++eCnt;
                break;
            case 'i':
            case 'I':

```

```

        ++iCnt;
        break;
    case 'o':
    case 'O':
        ++oCnt;
        break;
    case 'u':
    case 'U':
        ++uCnt;
        break;
    case ' ':
        ++spaceCnt;
        break;
    case '\t':
        ++tabCnt;
        break;
    case '\n':
        ++newlineCnt;
        break;
    }
}

cout << "元音字母 a 的数量是: " << aCnt << endl;
cout << "元音字母 e 的数量是: " << eCnt << endl;
cout << "元音字母 i 的数量是: " << iCnt << endl;
cout << "元音字母 o 的数量是: " << oCnt << endl;
cout << "元音字母 u 的数量是: " << uCnt << endl;
cout << "空格的数量是: " << spaceCnt << endl;
cout << "制表符的数量是: " << tabCnt << endl;
cout << "换行符的数量是: " << newlineCnt << endl;
return 0;
}

```

练习 5.12: 修改统计元音字母的程序，使其能统计以下含有两个字符的字符序列的数量：ff、fl 和 fi。

【出题思路】

为了统计字符序列的情况，必须记录前一个字符的内容。

【解答】

我们的设定是一个字符只会被统计一次。如果用户输入的序列是 xxxxxfflxxx，则统计结果是 ff：1 次、fl：0 次、fi：0 次。如果用户输入的序列是 xxxxxfifffflxxx，则统计结果是 ff：2 次、fl：1 次、fi：1 次。满足题意的程序如下所示：

```

#include <iostream>
using namespace std;

int main()
{
    unsigned int ffCnt = 0, flCnt = 0, fiCnt = 0;
    char ch, prech = '\0';

```

```

cout << "请输入一段文本: " << endl;
while(cin >> ch)
{
    bool bl = true;
    if(prech == 'f')
    {
        switch(ch)
        {
            case 'f':
                ++ffCnt;
                bl = false;
                break;
            case 'l':
                ++flCnt;
                break;
            case 'i':
                ++fiCnt;
                break;
        }
    }
    if(!bl)
        prech = '\0';
    else
        prech = ch;
}
cout << "ff 的数量是: " << ffCnt << endl;
cout << "fl 的数量是: " << flCnt << endl;
cout << "fi 的数量是: " << fiCnt << endl;
return 0;
}

```

练习 5.13: 下面显示的每个程序都含有一个常见的编程错误，指出错误在哪里，然后修改它们。

- (a) `unsigned aCnt = 0, eCnt = 0, iouCnt = 0;`
`char ch = next_text();`
`switch (ch) {`
`case 'a': aCnt++;`
`case 'e': eCnt++;`
`default: iouCnt++;`
`}`
- (b) `unsigned index = some_value();`
`switch (index) {`
`case 1:`
`int ix = get_value();`
`ivec[ ix ] = index;`
`break;`
`default:`
`ix = ivec.size()-1;`
`ivec[ ix ] = index;`
`}`

```

(c)  unsigned evenCnt = 0, oddCnt = 0;
    int digit = get_num() % 10;
    switch (digit) {
        case 1, 3, 5, 7, 9:
            oddcnt++;
            break;
        case 2, 4, 6, 8, 10:
            evencnt++;
            break;
    }

(d)  unsigned ival=512, jval=1024, kval=4096;
    unsigned bufsize;
    unsigned swt = get_bufCnt();
    switch(swt) {
        case ival:
            bufsize = ival * sizeof(int);
            break;
        case jval:
            bufsize = jval * sizeof(int);
            break;
        case kval:
            bufsize = kval * sizeof(int);
            break;
    }

```

【出题思路】

switch 语句有几个语法要点：必须在必要的地方使用 break; 语句，应该把变量定义在块作用域内，case 标签只能有一个值且不能是变量。

【解答】

(a)的错误是在每个 case 分支中都缺少了 break; 语句，造成的后果是一旦执行了前面的 case 分支，必定还会继续执行接下来的其他 case 分支。举例说明，如果 ch 的内容是字符'a'，则 aCnt、eCnt 和 iouCnt 的值都会增加；如果 ch 的内容是字符'e'，则 eCnt 和 iouCnt 的值都会增加，这显然与程序的预期是不相符的。

修改后的程序如下所示：

```

unsigned aCnt = 0, eCnt = 0, iouCnt = 0;
char ch = next_text();
switch (ch) {
    case 'a':
        aCnt++;
        break;
    case 'e':
        eCnt++;
        break;
    default:
        iouCnt++;
        break;
}

```

(b)的错误是在 case 分支中定义并初始化了变量 ix，同时在 default 分支中使用了该变量，此时如果控制流跳过 case 分支而直接到达 default 分支，则会试图使用未经初始化的变量，因而该程序无法通过编译。解决办法是，把 ix 的定义放

置在 switch 语句之前。

修改后的程序如下所示：

```
unsigned index = some_value();
int ix;
switch (index) {
    case 1:
        ix = get_value();
        ivec[ ix ] = index;
        break;
    default:
        ix = ivec.size()-1;
        ivec[ ix ] = index;
}
```

(c)的错误是在同一个 case 标签中放置了多个值，而 C++规定一个 case 标签只能对应一个值。修改后的程序如下所示：

```
unsigned evenCnt = 0, oddCnt = 0;
int digit = get_num() % 10;
switch (digit) {
    case 1:
    case 3:
    case 5:
    case 7:
    case 9:
        oddcnt++;
        break;
    case 2:
    case 4:
    case 6:
    case 8:
    case 10:
        evencnt++;
        break;
}
```

(d)的错误是使用变量作为 case 标签的内容，C++规定，case 标签的内容只能是整型常量表达式。修改后的程序如下所示：

```
const unsigned ival=512, jval=1024, kval=4096;
unsigned bufsize;
unsigned swt = get_bufCnt();
switch(swt) {
    case ival:
        bufsize = ival * sizeof(int);
        break;
    case jval:
        bufsize = jval * sizeof(int);
        break;
    case kval:
        bufsize = kval * sizeof(int);
        break;
}
```

练习 5.14: 编写一段程序, 从标准输入中读取若干 `string` 对象并查找连续重复出现的单词。所谓连续重复出现的意思是: 一个单词后面紧跟着这个单词本身。要求记录连续重复出现的最大次数以及对应的单词。如果这样的单词存在, 输出重复出现的最大次数; 如果不存在, 输出一条信息说明任何单词都没有连续出现过。例如, 如果输入是

```
how now now now brown cow cow
```

那么输出应该表明单词 `now` 连续出现了 3 次。

【出题思路】

使用 `while` 循环遍历用户输入的每一个字符串, 根据当前字符串与上一个字符串是否相等决定如何更新各个变量的状态。如果相等, 判断当前字符串连续出现的次数是否已经超过了系统记录的值, 如果超过则及时更新。

【解答】

满足题意的程序如下所示:

```
#include <iostream>
#include <string>
using namespace std;

int main()
{
    // 定义 3 个字符串变量, 分别表示:
    // 当前操作的字符串、前一个字符串、当前出现次数最多的字符串
    string currString, preString = "", maxString;
    // 定义 2 个整型变量, 分别表示:
    // 当前连续出现的字符串数量、当前出现次数最多的字符串数量
    int currCnt = 1, maxCnt = 0;
    while(cin >> currString)    // 检查每个字符串
    {
        // 如果当前字符串与前一个字符串一致, 更新状态
        if(currString == preString)
        {
            ++currCnt;
            if(currCnt > maxCnt)
            {
                maxCnt = currCnt;
                maxString = currString;
            }
        }
        // 如果当前字符串与前一个字符串不一致, 重置 currCnt
        else
        {
            currCnt = 1;
        }
        // 更新 preString 以便于下一次循环使用
        preString = currString;
    }
    if(maxCnt > 1)
        cout << "出现最多的字符串是: " << maxString
```

```

        << "，次数是：" << maxCnt << endl;
    else
        cout << "每个字符串都只出现了一次" << endl;
    return 0;
}

```

练习 5.15: 说明下列循环的含义并改正其中的错误。

```

(a)  for (int ix = 0; ix != sz; ++ix) { /* ... */ }
      if (ix != sz)
          // ...
(b)  int ix;
      for (ix != sz; ++ix) { /* ... */ }
(c)  for (int ix = 0; ix != sz; ++ix, ++sz) { /* ... */ }

```

【出题思路】

理解 for 循环的语法规则，特别是 for 循环控制结构中三条语句的作用。理解循环控制变量与循环终止条件的关系，从而能够判断循环是否会无限执行下去。

【解答】

(a) 的错误是在 for 语句中定义了变量 ix，然后试图在 for 语句之外继续使用 ix。因为 ix 定义在 for 语句的内部，所以其作用域仅限于 for 语句。在 if 语句中 ix 已经失效，因此程序无法编译通过。

修改后的程序如下：

```

int ix;
for (ix = 0; ix != sz; ++ix) { /* ... */ }
if (ix != sz)
    // ...

```

(b) 的错误有两个，一是变量 ix 未经初始化就直接使用，二是 for 语句的控制结构缺少一句话，在语法上是错误的。

修改后的程序如下：

```

int ix;
for (ix = 0; ix != sz; ++ix) { /* ... */ }

```

(c) 的错误是一旦进入循环，程序就会无休止地执行下去。也就是说，当初始情况下 ix != sz 时，由题意可知 ix 和 sz 一直同步增长，循环的终止条件永远不会满足，所以该循环是一个死循环。

修改后的程序如下：

```

for (int ix = 0; ix != sz; ++ix) { /* ... */ }

```

练习 5.16: while 循环特别适用于那种条件保持不变、反复执行操作的情况，例如，当未达到文件末尾时不断读取下一个值。for 循环则更像是在按步骤迭代，它的索引值在某个范围内依次变化。根据每种循环的习惯用法各自编写一段程序，然后分别用另一种循环改写。如果只能使用一种循环，你倾向于使用哪种呢？为什么？

【出题思路】

一般情况下，while 循环和 for 循环可以互相转换，不论哪种循环形式关键点

都有三个：循环控制变量的初始值、循环控制变量的变化规则、循环终止条件。编写循环语句时除了要满足应用需求，还必须时刻关注循环控制变量和循环终止条件，谨防写出死循环。

【解答】

从标准输入流读取数据的程序一般使用 while 循环，其形式是：

```
char ch;
while(cin >> ch)
{
    /* ... */
}
```

因为这项功能事实上不太要求我们严格跟踪循环变量的变化过程，所以改写成 for 循环后稍显冗余：

```
for( ; cin >> ch; )
{
    /* ... */
}
```

请注意：在上面的 for 循环控制结构中，第一条语句和第三条语句都是空语句，额外添加一个循环控制变量（比如 int i）是没有意义的。

整数累加求和的程序一般使用 for 循环，其形式是：

```
int iCount = 0;
for(int i = 0; i < 10; ++i)
{
    iCount += i;
}
```

使用 while 循环改写后的形式是：

```
int iCount = 0, i = 0;
while(i < 10)
{
    iCount += i;
    ++i;
}
```

在大多数情况下，两种循环形式可以互相转换。如果只能使用一种循环，作者倾向于使用 for 循环。for 循环的优点是结构严谨，便于控制程序的逻辑。

练习 5.17：假设有两个包含整数的 vector 对象，编写一段程序，检验其中一个 vector 对象是否是另一个的前缀。为了实现这一目标，对于两个不等长的 vector 对象，只需挑出长度较短的那个，把它的所有元素和另一个 vector 对象比较即可。例如，如果两个 vector 对象的元素分别是 0、1、1、2 和 0、1、1、2、3、5、8，则程序的返回结果应该为真。

【出题思路】

使用 while 循环实现两个 vector 对象的逐元素比较。

【解答】

满足题意的程序如下所示：

```

#include <iostream>
#include <vector>
using namespace std;

int main()
{
    vector<int> v1 = {0, 1, 1, 2};
    vector<int> v2 = {0, 1, 1, 2, 3, 5, 8};
    vector<int> v3 = {3, 5, 8};
    vector<int> v4 = {3, 5, 0, 9, 2, 7};

    auto it1 = v1.cbegin();           // 定义 v1 的迭代器
    auto it2 = v2.cbegin();           // 定义 v2 的迭代器
    auto it3 = v3.cbegin();           // 定义 v3 的迭代器
    auto it4 = v4.cbegin();           // 定义 v4 的迭代器
    // 设定循环条件: v1 和 v2 都尚未检查完
    while(it1 != v1.cend() && it2 != v2.cend())
    {
        // 如果当前位置的两个元素不相等, 则肯定没有前缀关系, 退出循环
        if(*it1 != *it2)
        {
            cout << "v1 和 v2 之间不存在前缀关系" << endl;
            break;
        }
        ++it1;                         // 迭代器移动到下一个元素
        ++it2;                         // 迭代器移动到下一个元素
    }
    if(it1 == v1.cend())                // 如果 v1 较短
    {
        cout << "v1 是 v2 的前缀" << endl;
    }
    if(it2 == v2.cend())                // 如果 v2 较短
    {
        cout << "v2 是 v1 的前缀" << endl;
    }
    return 0;
}

```

程序的输出结果是: v1 是 v2 的前缀。如果更新程序使其处理 v3 和 v4, 则程序将显示: v3 和 v4 之间不存在前缀关系。

练习 5.18: 说明下列循环的含义并改正其中的错误。

```

(a) do
    int v1, v2;
    cout << "Please enter two numbers to sum:" ;
    if (cin >> v1 >> v2)
        cout << "Sum is: " << v1 + v2 << endl;
    while (cin);

(b) do {
    // . . .

```

```

    } while (int ival = get_response());
(c) do {
    int ival = get_response();
    } while (ival);

```

【出题思路】

理解 do-while 循环的语法规则。do-while 与 while 的最主要区别是：do-while 语句至少会执行一次循环体；而 while 有可能一次都不执行循环体。

【解答】

(a)的含义是每次循环读入两个整数并输出它们的和。因为 do-while 语句的循环体必须是一条语句或者一个语句块，所以在本题中应该把循环体的内容用花括号括起来。修改后的程序是：

```

do{
    int v1, v2;
    cout << "Please enter two numbers to sum:" ;
    if (cin >> v1 >> v2)
        cout << "Sum is: " << v1 + v2 << endl;
}while (cin);

```

(b)的含义是当 get_response 的返回值不为 0 时执行循环体。因为 do-while 语句不允许在循环条件内定义变量，所以该程序是错误的。修改后的程序是：

```

int ival;
do {
    ival = get_response();
} while (ival);

```

(c) 的含义是当 get_response 的返回值不为 0 时执行循环体。因为出现在 do-while 语句条件部分的变量必须定义在循环体之外，所以该程序是错误的。修改后的程序是：

```

int ival;
do {
    ival = get_response();
} while (ival);

```

练习 5.19：编写一段程序，使用 do-while 循环重复地执行下述任务：首先提示用户输入两个 string 对象，然后挑出较短的那个并输出它。

【出题思路】

把题目要求的任务写在 do-while 循环体中即可，该程序至少会执行一次比较操作。把字符串定义成 string 对象，调用 size 函数即可得到 string 对象的长度。

【解答】

满足题意的程序如下所示：

```

#include <iostream>
#include <string>
using namespace std;

```

```

int main()
{
    do{

```

```

        cout << "请输入两个字符串: " << endl;
        string str1, str2;
        cin >> str1 >> str2;
        if(str1.size() < str2.size())
            cout << "长度较小的字符串是: " << str1 << endl;
        else if(str1.size() > str2.size())
            cout << "长度较小的字符串是: " << str2 << endl;
        else
            cout << "两个字符串等长" << endl;
    }while (cin);

    return 0;
}

```

练习 5.20: 编写一段程序，从标准输入中读取 `string` 对象的序列直到连续出现两个相同的单词或者所有单词都读完为止。使用 `while` 循环一次读取一个单词，当一个单词连续出现两次时使用 `break` 语句终止循环。输出连续重复出现的单词，或者输出一个消息说明没有任何单词是连续重复出现的。

【出题思路】

`break` 语句是一种跳转语句，负责终止离它最近的 `while`、`do-while`、`for` 和 `switch` 语句，并把程序的控制权交给这些语句之后的第一条语句。

【解答】

满足题意的程序如下所示：

```

#include <iostream>
#include <string>
using namespace std;

int main()
{
    string currString, preString;
    bool b1 = true;
    cout << "请输入一组字符串: " << endl;
    while(cin >> currString)
    {
        if(currString == preString)
        {
            b1 = false;
            cout << "连续出现的字符串是: " << currString << endl;
            break;
        }
        preString = currString;
    }
    if(b1)
        cout << "没有连续出现的字符串" << endl;
    return 0;
}

```

本题用到了一个起标识作用的布尔值 `b1`，当 `b1` 为真时表示没有连续出现的字符串，一旦我们发现了存在连续出现的字符串，就立即把 `b1` 的值置为 `false`。

练习 5.21: 修改 5.5.1 节 (第 171 页) 练习题的程序, 使其找到的重复单词必须以大写字母开头。

【出题思路】

`continue` 语句是一种跳转语句, 负责终止最近的循环中的当前迭代并立即开始下一次迭代。与 `break` 语句的区别是, `continue` 虽然终止了当前迭代, 但是并不终止循环; 而 `break` 语句则直接跳出循环。

【解答】

根据题目的要求, 我们首先检查读入的单词是否以大写字母开头, 如果不是则终止当前迭代并开始下一次循环。满足题意的程序是:

```
#include <iostream>
#include <string>
using namespace std;

int main()
{
    string currString, preString;
    bool bl = true;
    cout << "请输入一组字符串: " << endl;
    while(cin >> currString)
    {
        if(!isupper(currString[0]))
            continue;
        if(currString == preString)
        {
            bl = false;
            cout << "连续出现的字符串是: " << currString << endl;
            break;
        }
        preString = currString;
    }
    if(bl)
        cout << "没有连续出现的字符串" << endl;
    return 0;
}
```

练习 5.22: 本节的最后一个例子跳回到 `begin`, 其实使用循环能更好地完成该任务。重写这段代码, 注意不再使用 `goto` 语句。

【出题思路】

`goto` 语句的作用是从 `goto` 语句无条件跳转到同一函数的另一条语句。但是一般情况下, 不建议读者使用 `goto` 语句, 因为它使得程序既难理解又难修改。

【解答】

用 `do-while` 语句改写后的程序如下所示:

```
int sz;
do{
    sz = get_size();
}while(sz <= 0);
```

练习 5.23：编写一段程序，从标准输入读取两个整数，输出第一个数除以第二个数的结果。

【出题思路】

对于整数除法来说，我们必须检查除数是否为 0。在这个版本的程序中，暂时不使用 try-catch 机制，而是直接输出错误信息。

【解答】

满足题意的程序如下所示：

```
#include <iostream>
using namespace std;

int main()
{
    cout << "请依次输入被除数和除数：" << endl;
    int ival1, ival2;
    cin >> ival1 >> ival2;
    if(ival2 == 0)
    {
        cout << "除数不能为 0" << endl;
        return -1;
    }
    cout << "两数相除的结果是：" << ival1 / ival2 << endl;
    return 0;
}
```

练习 5.24：修改你的程序，使得当第二个数是 0 时抛出异常。先不要设定 catch 子句，运行程序并真的为除数输入 0，看看会发生什么？

【出题思路】

与上一版本相比，通过抛出异常来提示用户的输入错误。

【解答】

满足题意的程序如下所示：

```
#include <iostream>
#include <stdexcept>
using namespace std;

int main()
{
    cout << "请依次输入被除数和除数：" << endl;
    int ival1, ival2;
    cin >> ival1 >> ival2;
    if(ival2 == 0)
    {
        throw runtime_error("除数不能为 0");
    }
    cout << "两数相除的结果是：" << ival1 / ival2 << endl;
    return 0;
}
```

在本题中，我们设定当检测到除数为 0 时抛出一个 `runtime_error` 异常，因为没有 `catch` 语句，所以系统只报告异常而并不处理它。在我们的编译环境中，系统给出的报错信息是：

```
terminate called after throwing an instance of 'std::runtime_error'
what(): 除数不能为 0
This application has requested the Runtime to terminate it in an unusual
way. Please contact the application's support team for more information.
```

练习 5.25: 修改上一题的程序，使用 `try` 语句块捕获异常。`catch` 子句应该为用户输出一条提示信息，询问其是否输入新数并重新执行 `try` 语句块的内容。

【出题思路】

使用 `try-catch` 结构实现完整的异常处理机制。

【解答】

满足题意的程序如下所示：

```
#include <iostream>
#include <stdexcept>
using namespace std;

int main()
{
    cout << "请依次输入被除数和除数: " << endl;
    int ival1, ival2;
    while(cin >> ival1 >> ival2)
    {
        try
        {
            if(ival2 == 0)
            {
                throw runtime_error("除数不能为 0");
            }
            cout << "两数相除的结果是: " << ival1 / ival2 << endl;
        } catch(runtime_error err)
        {
            cout << err.what() << endl;
            cout << "需要继续吗 (y or n)? ";
            char ch;
            cin >> ch;
            if(ch != 'y' && ch != 'Y')
                break;
        }
    }

    return 0;
}
```